

Header & Notification Panel

▼ Relevant source files

Frontend/STUDER/src/app/features/notifications/notification.model.ts

Frontend/STUDER/src/app/features/notifications/notification.service.ts

Frontend/STUDER/src/app/shared/components/header/header.component.css

Frontend/STUDER/src/app/shared/components/header/header.component.html

Frontend/STUDER/src/app/shared/components/header/header.component.ts

Frontend/STUDER/src/app/shared/components/notification-panel/notification-panel.component.html

Frontend/STUDER/src/app/shared/components/notification-panel/notification-panel.component.ts

The `HeaderComponent` and `NotificationPanelComponent` provide the primary navigational and informational shell for the STUDER platform. They manage global concerns such as user search, system theming, language localization, and real-time notification tracking.

Header Component (studer-header)

The `HeaderComponent` acts as the top-level navigation bar. It integrates several core services to provide a reactive user experience.

Reactive Search Pipeline

The header implements a debounced search pipeline to prevent excessive API calls while the user types `Frontend/STUDER/src/app/shared/components/header/header.component.ts` 63-86

- **Debounce:** Waits 250ms after the last keystroke before triggering a request

`Frontend/STUDER/src/app/shared/components/header/header.component.ts` 65

- **DistinctUntilChanged:** Ensures requests are only sent if the query text has actually changed `Frontend/STUDER/src/app/shared/components/header/header.component.ts` 66

- **SwitchMap:** Cancels previous pending search requests if a new one is initiated

`Frontend/STUDER/src/app/shared/components/header/header.component.ts` 67

- **Implementation:** Uses `userService.searchUsers` to fetch a preview of up to 5 users

Frontend/STUDER/src/app/shared/components/header/header.component.ts 77

Global Controls

- **Theme Toggle:** Switches between `light` and `dark` modes by calling

`ThemeService.setTheme()`

Frontend/STUDER/src/app/shared/components/header/header.component.ts 98-101

- **Language Toggle:** Toggles between `en` and `es` via `LanguageService.setLanguage()`

Frontend/STUDER/src/app/shared/components/header/header.component.ts 107-110

- **Logout:** Invokes `AuthService.logout()` and redirects the user to the `/login` route

Frontend/STUDER/src/app/shared/components/header/header.component.ts 116-123

Notification Badge

The header maintains a reactive `unreadCount` by subscribing to

`NotificationService.unreadCount$`

Frontend/STUDER/src/app/shared/components/header/header.component.ts 57-59

The UI displays a

badge that caps at "99+" for values exceeding 99

Frontend/STUDER/src/app/shared/components/header/header.component.html 86-88

Sources: Frontend/STUDER/src/app/shared/components/header/header.component.ts 34-164

Frontend/STUDER/src/app/shared/components/header/header.component.html 1-96

Notification Panel

The `NotificationPanelComponent` is rendered inside an overlay triggered by the bell icon in the header

Frontend/STUDER/src/app/shared/components/header/header.component.html 83-96

Data Fetching and Filtering

The panel supports two view modes defined by the `NotificationFilter` type: `ALL` or `UNREAD`

Frontend/STUDER/src/app/shared/components/notification-panel/notification-panel.component.ts

13

- **Pagination:** Uses `loadMore()` to increment `currentPage` and append new results to the existing list

Frontend/STUDER/src/app/shared/components/notification-panel/notification-panel.component.ts

66-69

- **Filtering:** When the filter is set to `UNREAD`, the component passes `read: false` to the `NotificationService`

Frontend/STUDER/src/app/shared/components/notification-panel/notification-panel.component.ts

47-50

Notification Interactions

1. **Mark as Read:** Users can click a checkmark icon to mark a notification as read without navigating

Frontend/STUDER/src/app/shared/components/notification-panel/notification-panel.component.html

46-53

This triggers `decrementUnread()` in the service to update the header badge immediately

Frontend/STUDER/src/app/shared/components/notification-panel/notification-panel.component.ts

87

2. **Navigation:** Clicking a notification body triggers `navigateTo()`. This function marks the notification as read and uses the `NotificationStrategyFactory` to determine the correct route based on the `LinkType` (e.g., `DISCUSSION`, `MESSAGE`, `USER`)

Frontend/STUDER/src/app/shared/components/notification-panel/notification-panel.component.ts

92-104

Sources:

Frontend/STUDER/src/app/shared/components/notification-panel/notification-panel.component.ts

15-109

Frontend/STUDER/src/app/shared/components/notification-panel/notification-panel.component.html

1-76

Notification Service & Logic Flow

The `NotificationService` manages the state of unread notifications and provides a bridge to the strategy-based routing logic.

Unread Count Synchronization

The service uses a `BehaviorSubject` to broadcast the current unread count across the application

Frontend/STUDER/src/app/features/notifications/notification.service.ts

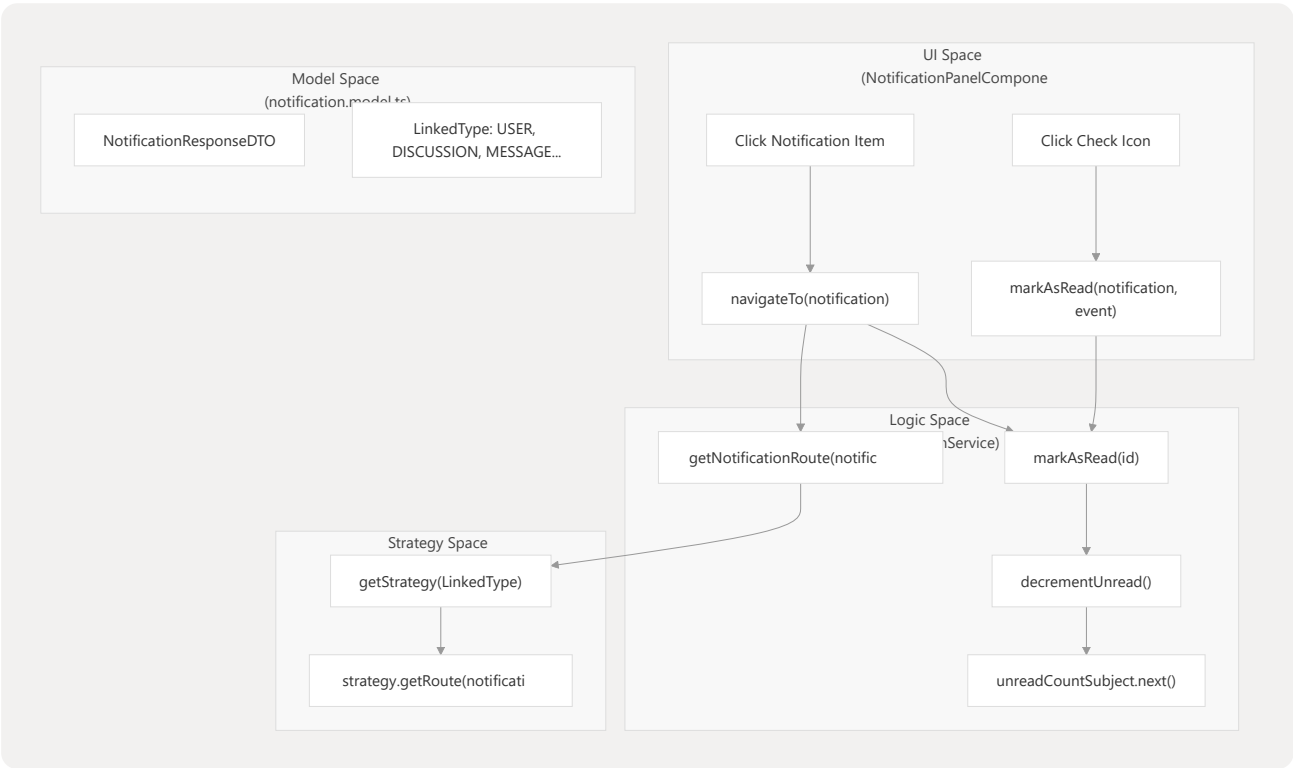
15-16

Method	Description
<code>loadUnreadCount()</code>	Fetches the first page of unread notifications to initialize the badge Frontend/STUDER/src/app/features/notifications/notification.service.ts 54-59
<code>decrementUnread()</code>	Subtracts 1 from the current <code>BehaviorSubject</code> value if greater than 0 Frontend/STUDER/src/app/features/notifications/notification.service.ts 61-66
<code>markAsRead(id)</code>	Sends a <code>PATCH</code> request to <code>/api/v1/notifications/{id}/read</code> Frontend/STUDER/src/app/features/notifications/notification.service.ts 47-52

Entity Mapping Diagram

This diagram maps the UI interaction in the `NotificationPanelComponent` to the underlying service and model entities.

Notification Action Flow



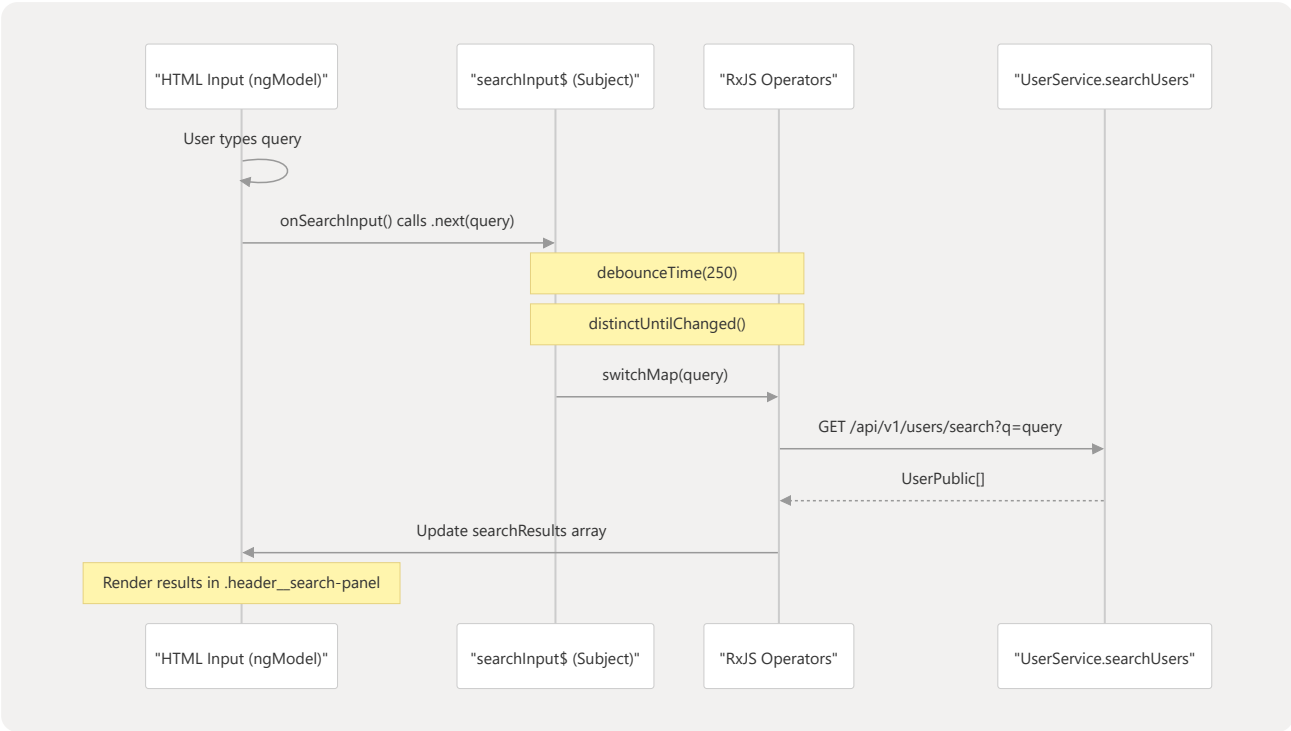
Sources: Frontend/STUDER/src/app/features/notifications/notification.service.ts 14-82

Frontend/STUDER/src/app/features/notifications/notification.model.ts 1-18

Search Pipeline Data Flow

The following diagram details the reactive search implementation within the **HeaderComponent** .

Search Data Flow



Sources: Frontend/STUDER/src/app/shared/components/header/header.component.ts 63-87

Frontend/STUDER/src/app/shared/components/header/header.component.html 10-23